

LC 2195 → Append K integers with minimal sum

Authored by: **Arpit Raj, LNMIIT Jaipur**

Date: September 19, 2026

Target: Amazon, Google

Example:

[1, 4, 25, 10, 25], k = 2

Then (2, 3) are missing in array. We can simply add them.

(2 + 3) = 5 → ans

Bruteforce Method

This Bruteforce method of starting curr from 1 and checking if it exist in array → then add to ans curr++, else curr++...

This will give TLE.

```
class Solution { public: long long minimalKSum(vector<int>& nums, int k) {
    int n=nums.size(); unordered_set<int> st; for(int i=0;i<n;i++){
        st.insert(nums[i]); } long long ans=0; int aadya=1; int cnt=0; while(cnt<k){
        if(st.find(aadya)==st.end()){ ans+=aadya; cnt++; } aadya++; //aadya to any case me
        badhega //maan lo [2,3,4] //initially 1 nhi mila to ans+=1; //now aadya=2; //ab 2
        already he to 3 try karo 4 try karo 5 try karo so on //to aadya else case me bhi
        badhegi } return ans; } };
```

Option 2 (Optimal Approach)

Let curr be **x**, and nums[i] be **y**.
Then everything bw **x** and **y** is our missing numbers.
We need to add **x, x+1, x+2, ..., y-1**

cnt = min(cnt, k)

Sequence to add: (**x, x+1, x+2, ... (x + cnt - 1)**)

$$\text{AP sum formula} = \frac{n}{2} [2a + (n-1) \times d]$$

Where:

n = cnt

a = x = curr

d = 1

Handling Remaining Elements

Now it's possible even after traversing array, still some **k** are left.

Eg: [1, 3, 4, 5], **k** = 3

You filled with 2.

Updated curr = nums[i] + 1.

Eventually reached 6.

Now need to add 6, 7.

So for the remaining **k**:

n = **k** (remaining **k**)

a = curr

d = 1

Step-by-Step Logic Summary

- sort
- ans = 0, curr = 1
- for (int i=0; i<n && k>0; i++)
- cnt = nums[i] - curr
- cnt = min(cnt, k) → *jitne ki jarurat sirf utna*

Optimized Code Implementation

```
class Solution { public: long long minimalKSum(vector<int>& nums, int k) {
    int n=nums.size(); sort(nums.begin(),nums.end()); long long ans=0; int aadya=1;
    for(int i=0;i<n && k>0;i++){ int cnt=nums[i]-aadya; cnt=min(cnt,k); if(cnt<0){
        continue; } //need to do sum of //aadya,aadya+1,aadya+2....aadya+cnt-1 //because
        these can fit in gap //AP sum formula //n/2*(2a+(n-1)*d) //n=cnt; //a=aadya //d=1;
        long long sum=(long long)cnt*(2*aadya+(cnt-1))/2; ans+=sum; k-=cnt;
        aadya=nums[i]+1; //next candidate } //for leftover required k //n=k; //a=aadya
        //d=1; if(k>0){ long long sum=(long long) k*(2*aadya+(k-1))/2; ans+=sum; } return
        ans; } };
```